Claim Of Core Client/Server Code Reference

이 문서는 클라이언트와 서버 프로젝트의 주요 소스 파일을 기준으로, 프로젝트 내부에서 정의된 함수와 멤버 변수를 파일별로 정리한 참조 문서입니다.

- 생성 기준: 프로젝트 소스 파일의 선언/정의 자동 추출 + 이름 기반 기능 설명
- 용도: 이후 GPT에게 구조 설명, 수정 위치 질문, 네트워크 흐름 질문을 할 때 참조 자료로 사용
- 주의: 일부 설명은 함수/변수명과 위치를 바탕으로 한 요약 설명이며, 엔진 내부 동작까지 완전히 대체하지는 않습니다.

문서 구성

- 클라이언트 파일 수: 26
- 서버 파일 수: 11
- 전체 파일 수: 37

클라이언트

BaseItem.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩BaseItem.h`
- 파일 목적: 아이템 기본 동작과 줍기/버리기 상호작용 기반을 담당하는 파일입니다.
- 함수 수(추출): 10
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void ABaseItem::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
SetOwnerCharacter	void ABaseItem::SetOwnerCharacter(AMyCharacter* NewOwner)	Owner Character 값을 현재 객체에 설정합니다.
OnBeginOverlap	void ABaseItem::OnBeginOverlap(UPrimitiveComponent* OverlappedComp, AActor* OtherActor, UPrimitiveComponent* OtherComp, int32 OtherBodyIndex, bool bFromSweep, const FHitResult& SweepResult)	On Begin Overlap 기능을 수행하는 함수입니다.
OnEndOverlap	void ABaseItem::OnEndOverlap(UPrimitiveComponent* OverlappedComp, AActor* OtherActor, UPrimitiveComponent* OtherComp, int32 OtherBodyIndex)	On End Overlap 기능을 수행하는 함수입니다.
StartUse	void ABaseItem::StartUse()	Start Use 기능을 수행하는 함수입니다.
DoHit	void ABaseItem::DoHit()	Do Hit 기능을 수행하는 함수입니다.
ApplyDamage	void ABaseItem::ApplyDamage(AActor* Target)	Damage 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
ApplyKnockback	void ABaseItem::ApplyKnockback(ACharacter* Target)	Knockback 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
OnStartUse	void ABaseItem::OnStartUse()	On Start Use 기능을 수행하는 함수입니다.
OnHitTarget	void ABaseItem::OnHitTarget(AActor* Target)	On Hit Target 기능을 수행하는 함수입니다.

Base₩Claim_Of_Core.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Base₩Claim_Of_Core.h`
- 파일 목적: 게임 모듈, 플레이어 컨트롤러 등 엔진 진입점과 기본 클래스를 담당하는 파일입니다.
- 함수 수(추출): 0
- 멤버 변수 수(추출): 0

Base₩Claim_Of_CorePlayerController.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Base₩Claim_Of_CorePlayerController.h`
- 파일 목적: 게임 모듈, 플레이어 컨트롤러 등 엔진 진입점과 기본 클래스를 담당하는 파일입니다.
- 함수 수(추출): 2
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void AClaim_Of_CorePlayerController::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
SetupInputComponent	void AClaim_Of_CorePlayerController::SetupInputComponent()	up Input Component 값을 현재 객체에 설정합니다.

CharacterInfoWidget.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩CharacterInfoWidget.h`
- 파일 목적: 클라이언트 측 CharacterInfoWidget 기능을 정의하는 파일입니다.
- 함수 수(추출): 6
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
NativeConstruct	void UCharacterInfoWidget::NativeConstruct()	Native Construct 기능을 수행하는 함수입니다.
NativeDestruct	void UCharacterInfoWidget::NativeDestruct()	Native Destruct 기능을 수행하는 함수입니다.
OnStrikerClicked	void UCharacterInfoWidget::OnStrikerClicked()	On Striker Clicked 기능을 수행하는 함수입니다.
OnGuardianClicked	void UCharacterInfoWidget::OnGuardianClicked()	On Guardian Clicked 기능을 수행하는 함수입니다.
OnManipulatorClicked	void UCharacterInfoWidget::OnManipulatorClicked()	On Manipulator Clicked 기능을 수행하는 함수입니다.
HandleRoleSelected	void UCharacterInfoWidget::HandleRoleSelected(int32 InRoleType)	Role Selected 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.

GameState₩InGame_GameState.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩GameState₩InGame_GameState.h`
- 파일 목적: 인게임 공용 상태와 라운드 흐름 표시를 담당하는 파일입니다.
- 함수 수(추출): 7
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void AInGame_GameState::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
StartReady	void AInGame_GameState::StartReady()	Start Ready 기능을 수행하는 함수입니다.
StartRound	void AInGame_GameState::StartRound()	Start Round 기능을 수행하는 함수입니다.
EndRound	void AInGame_GameState::EndRound()	End Round 기능을 수행하는 함수입니다.
CountdownTick	void AInGame_GameState::CountdownTick()	Countdown Tick 기능을 수행하는 함수입니다.
ActivateGameplayFromServerStart	void AInGame_GameState::ActivateGameplayFromServerStart()	Activate Gameplay From Server Start 기능을 수행하는 함수입니다.
UpdatePhase	void AInGame_GameState::UpdatePhase()	Phase 상태나 표시값을 최신 값으로 갱신합니다.

Map₩BreakableActor.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩BreakableActor.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 7
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void ABreakableActor::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
ActivateBreakable	void ABreakableActor::ActivateBreakable()	Activate Breakable 기능을 수행하는 함수입니다.
ResetBreakable	void ABreakableActor::ResetBreakable()	Reset Breakable 기능을 수행하는 함수입니다.
ApplyBreakableDamage	void ABreakableActor::ApplyBreakableDamage(float DamageAmount)	Breakable Damage 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
BreakActor	void ABreakableActor::BreakActor()	Break Actor 기능을 수행하는 함수입니다.
OnActivatedInternal	void ABreakableActor::OnActivatedInternal()	On Activated Internal 기능을 수행하는 함수입니다.
OnBrokenInternal	void ABreakableActor::OnBrokenInternal()	On Broken Internal 기능을 수행하는 함수입니다.

Map₩Building₩DebrisSpawner.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩Building₩DebrisSpawner.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 6
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void ADebrisSpawner::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
Tick	void ADebrisSpawner::Tick(float DeltaTime)	매 프레임 또는 주기적으로 호출되어 상태를 갱신합니다.
InitializeGameState	void ADebrisSpawner::InitializeGameState()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.

이름	시그니처	설명
UpdateSpawner	void ADebrisSpawner::UpdateSpawner(float DeltaTime)	Spawner 상태나 표시값을 최신 값으로 갱신합니다.
SpawnDebrisByCurrentPhase	void ADebrisSpawner::SpawnDebrisByCurrentPhase()	Spawn Debris By Current Phase 기능을 수행하는 함수입니다.
PostEditChangeProperty	if WITH_EDITOR void ADebrisSpawner::PostEditChangeProperty(FPropertyChangedEvent& PropertyChangedEvent)	Post Edit Change Property 기능을 수행하는 함수입니다.

Map₩Building₩LargeDebrisActor.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩Building₩LargeDebrisActor.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 12
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void ALargeDebrisActor::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
Tick	void ALargeDebrisActor::Tick(float DeltaTime)	매 프레임 또는 주기적으로 호출되어 상태를 갱신합니다.
PrepareDebris	void ALargeDebrisActor::PrepareDebris()	Prepare Debris 기능을 수행하는 함수입니다.
ActivateDebris	void ALargeDebrisActor::ActivateDebris(float InitialImpulseStrength)	Activate Debris 기능을 수행하는 함수입니다.
UpdateActorFall	void ALargeDebrisActor::UpdateActorFall(float DeltaTime)	Actor Fall 상태나 표시값을 최신 값으로 갱신합니다.
LandAndFracture	void ALargeDebrisActor::LandAndFracture()	Land And Fracture 기능을 수행하는 함수입니다.
BreakInitialBottomChunks	void ALargeDebrisActor::BreakInitialBottomChunks(float ImpactSpeed)	Break Initial Bottom Chunks 기능을 수행하는 함수입니다.
BreakChunk	void ALargeDebrisActor::BreakChunk(int32 ChunkIndex, bool bFromImpact)	Break Chunk 기능을 수행하는 함수입니다.
DropUnsupportedChunks	void ALargeDebrisActor::DropUnsupportedChunks()	Drop Unsupported Chunks 기능을 수행하는 함수입니다.
StartSequentialUnsupportedBreak	void ALargeDebrisActor::StartSequentialUnsupportedBreak()	Start Sequential Unsupported Break 기능을 수행하는 함수입니다.
ProcessNextChunkBreak	void ALargeDebrisActor::ProcessNextChunkBreak()	입력 데이터를 해석하고 적절한 처리 흐름으로 연결합니다.
OnChunkBrokenInternal	void ALargeDebrisActor::OnChunkBrokenInternal(int32 BrokenChunkIndex, bool bFromImpact)	On Chunk Broken Internal 기능을 수행하는 함수입니다.

Map₩Building₩LargeDebrisController.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩Building₩LargeDebrisController.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 10
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
HandleGameplayActivated	void ALargeDebrisController::HandleGameplayActivated()	Gameplay Activated 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
ActivateGameplayRuntime	void ALargeDebrisController::ActivateGameplayRuntime()	Activate Gameplay Runtime 기능을 수행하는 함수입니다.
BeginPlay	void ALargeDebrisController::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
Tick	void ALargeDebrisController::Tick(float DeltaTime)	매 프레임 또는 주기적으로 호출되어 상태를 갱신합니다.
InitializeGameState	void ALargeDebrisController::InitializeGameState()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
HandlePhaseLogic	void ALargeDebrisController::HandlePhaseLogic()	Phase Logic 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
TriggerPhase2Debris	void ALargeDebrisController::TriggerPhase2Debris()	Trigger Phase 2 Debris 기능을 수행하는 함수입니다.
TriggerPhase3Debris	void ALargeDebrisController::TriggerPhase3Debris()	Trigger Phase 3 Debris 기능을 수행하는 함수입니다.
PrepareDebrisActors	void ALargeDebrisController::PrepareDebrisActors(const TArray<TObjectPtr<ALargeDebrisActor>>& DebrisActors)	Prepare Debris Actors 기능을 수행하는 함수입니다.
ActivateDebrisActors	void ALargeDebrisController::ActivateDebrisActors(const TArray<TObjectPtr<ALargeDebrisActor>>& DebrisActors)	Activate Debris Actors 기능을 수행하는 함수입니다.

Map₩Building₩SmallDebrisActor.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩Building₩SmallDebrisActor.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 4
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void ASmallDebrisActor::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
OnDebrisHit	void ASmallDebrisActor::OnDebrisHit(UPrimitiveComponent* HitComponent, AActor* OtherActor, UPrimitiveComponent* OtherComp, FVector NormalImpulse, const FHitResult& Hit)	On Debris Hit 기능을 수행하는 함수입니다.
StartDestroyTimer	void ASmallDebrisActor::StartDestroyTimer()	Start Destroy Timer 기능을 수행하는 함수입니다.
DestroySelf	void ASmallDebrisActor::DestroySelf()	Destroy Self 기능을 수행하는 함수입니다.

Map₩IceCave₩IceFloorTile.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩IceCave₩IceFloorTile.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 4
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void AIceFloorTile::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
SetCracked	void AIceFloorTile::SetCracked()	Cracked 값을 현재 객체에 설정합니다.
StartBreaking	void AIceFloorTile::StartBreaking()	Start Breaking 기능을 수행하는 함수입니다.
BreakFloor	void AIceFloorTile::BreakFloor()	Break Floor 기능을 수행하는 함수입니다.

Map₩IceCave₩IceMapController.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩IceCave₩IceMapController.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 6
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void AIceMapController::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
StartIceMap	void AIceMapController::StartIceMap()	Start Ice Map 기능을 수행하는 함수입니다.
EnterPhase2	void AIceMapController::EnterPhase2()	Enter Phase 2 기능을 수행하는 함수입니다.
EnterPhase3	void AIceMapController::EnterPhase3()	Enter Phase 3 기능을 수행하는 함수입니다.
TriggerRandomIcicle	void AIceMapController::TriggerRandomIcicle()	Trigger Random Icicle 기능을 수행하는 함수입니다.
TriggerRandomFloorTile	void AIceMapController::TriggerRandomFloorTile()	Trigger Random Floor Tile 기능을 수행하는 함수입니다.

Map₩IceCave₩IcicleActor.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩IceCave₩IcicleActor.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 5
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void AIcicleActor::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
StartWarning	void AIcicleActor::StartWarning()	Start Warning 기능을 수행하는 함수입니다.
StartFalling	void AIcicleActor::StartFalling()	Start Falling 기능을 수행하는 함수입니다.
OnIcicleHit	void AIcicleActor::OnIcicleHit(UPrimitiveComponent* HitComponent, AActor* OtherActor, UPrimitiveComponent* OtherComp, FVector NormalImpulse, const FHitResult& Hit)	On Icicle Hit 기능을 수행하는 함수입니다.
BreakIcicle	void AIcicleActor::BreakIcicle()	Break Icicle 기능을 수행하는 함수입니다.

Map₩MapPhaseManager.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩MapPhaseManager.h`

- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 6
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void AMapPhaseManager::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
Tick	void AMapPhaseManager::Tick(float DeltaTime)	매 프레임 또는 주기적으로 호출되어 상태를 갱신합니다.
InitializeFromGameState	void AMapPhaseManager::InitializeFromGameState()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
UpdateFromGameState	void AMapPhaseManager::UpdateFromGameState()	From Game State 상태나 표시값을 최신 값으로 갱신합니다.
HandlePhaseChanged	void AMapPhaseManager::HandlePhaseChanged(EMapPhase NewPhase)	Phase Changed 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleRoundStateChanged	void AMapPhaseManager::HandleRoundStateChanged(ERoundState NewRoundState)	Round State Changed 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.

Map₩PreFracturedActor.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Map₩PreFracturedActor.h`
- 파일 목적: 맵 기믹, 지형 오브젝트, 파괴/빙판/잔해 등 환경 상호작용을 담당하는 파일입니다.
- 함수 수(추출): 15
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void APreFracturedActor::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
InitializeGameState	void APreFracturedActor::InitializeGameState()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
HandleGameplayActivated	void APreFracturedActor::HandleGameplayActivated()	Gameplay Activated 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
ActivateFractureRuntime	void APreFracturedActor::ActivateFractureRuntime()	Activate Fracture Runtime 기능을 수행하는 함수입니다.
CollectChunkMeshes	void APreFracturedActor::CollectChunkMeshes()	Collect Chunk Meshes 기능을 수행하는 함수입니다.
InitializeChunkData	void APreFracturedActor::InitializeChunkData()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
PrepareFracture	void APreFracturedActor::PrepareFracture()	Prepare Fracture 기능을 수행하는 함수입니다.
ApplyDamageToChunk	void APreFracturedActor::ApplyDamageToChunk(int32 ChunkIndex, float DamageAmount)	Damage To Chunk 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
BreakChunk	void APreFracturedActor::BreakChunk(int32 ChunkIndex, bool bFromImpact)	Break Chunk 기능을 수행하는 함수입니다.
OnChunkBrokenInternal	void APreFracturedActor::OnChunkBrokenInternal(int32 BrokenChunkIndex, bool bFromImpact)	On Chunk Broken Internal 기능을 수행하는 함수입니다.
DropUnsupportedChunks	void APreFracturedActor::DropUnsupportedChunks()	Drop Unsupported Chunks 기능을 수행하는 함수입니다.
FreezeRemainingChunks	void APreFracturedActor::FreezeRemainingChunks()	Freeze Remaining Chunks 기능을 수행하는 함수입니다.

이름	시그니처	설명
OnBrokenInternal	void APreFracturedActor::OnBrokenInternal()	On Broken Internal 기능을 수행하는 함수입니다.
AutoBuildNeighborGraph	void APreFracturedActor::AutoBuildNeighborGraph()	Auto Build Neighbor Graph 기능을 수행하는 함수입니다.
AutoSetAnchoredChunks	void APreFracturedActor::AutoSetAnchoredChunks()	Auto Set Anchored Chunks 기능을 수행하는 함수입니다.

Networking₩ClientNetworkTypes.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Networking₩ClientNetworkTypes.h`
- 파일 목적: 네트워크 연결, 패킷 전송/수신, 이벤트 분배를 담당하는 파일입니다.
- 주요 타입: `GameData, FPacketHeader, FLoginPacket, FRoomPacketData, FRoomMemberPacketData, FGamedataPacket, FMapEventPacket, FSpawnObjectPacket, FPhaseChangePacket, FMovePacket`
- 함수 수(추출): 0
- 멤버 변수 수(추출): 32

멤버 변수

이름	타입	설명
userUID	int	세션/플레이어 식별자와 관련된 상태입니다.
isConnected	bool	isConnected 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
maxHP	int	체력 관련 현재값/최대값을 저장합니다.
currentHP	int	체력 관련 현재값/최대값을 저장합니다.
x	float	x 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
rotate	float	rotate 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
baseWalkSpeed	float	기능 토글 또는 현재 상태를 나타내는 불리언 플래그입니다.
animationNum	int	animationNum 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
PacketSize	uint16	PacketSize 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
PacketID	uint16	PacketID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
userID	FString	userID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
userPW	FString	userPW 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
RoomID	int32	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
RoomName[20]	wchar_t	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
CurrentPlayers	int32	CurrentPlayers 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
PlayerName[20]	wchar_t	PlayerName[20] 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
isReady	bool	isReady 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
isHost	bool	isHost 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
sessionID	int	세션/플레이어 식별자와 관련된 상태입니다.
data	GameData	data 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

이름	타입	설명
eventType	int32_t	eventType 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
objectIndex	int32_t	objectIndex 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
eventState	int32_t	eventState 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
objectType	int32_t	objectType 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
z	float x, y,	z 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
objectID	int32_t	objectID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
roundState	int32_t	roundState 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
mapPhase	int32_t	맵, 레벨, 라운드 흐름과 관련된 참조 또는 상태값입니다.
gameTime	float	gameTime 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
Z	float X, Y,	Z 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
Yaw	float	Yaw 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
VelocityY	float VelocityX,	VelocityY 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

Networking₩ClientNetworkWorker.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Networking₩ClientNetworkWorker.h`
- 파일 목적: 네트워크 연결, 패킷 전송/수신, 이벤트 분배를 담당하는 파일입니다.
- 함수 수(추출): 26
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
Init	bool FClientNetworkWorker::Init()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
Run	uint32 FClientNetworkWorker::Run()	Run 기능을 수행하는 함수입니다.
Stop	void FClientNetworkWorker::Stop()	Stop 기능을 수행하는 함수입니다.
Exit	void FClientNetworkWorker::Exit()	Exit 기능을 수행하는 함수입니다.
WakeUp	void FClientNetworkWorker::WakeUp()	Wake Up 기능을 수행하는 함수입니다.
TryConnectBlocking	bool FClientNetworkWorker::TryConnectBlocking(const FString& IP, int32 Port)	조건을 확인한 뒤 요청 또는 동작을 시도합니다.
CloseSocket	void FClientNetworkWorker::CloseSocket()	Close Socket 기능을 수행하는 함수입니다.
FlushInboundCommands	void FClientNetworkWorker::FlushInboundCommands()	Flush Inbound Commands 기능을 수행하는 함수입니다.
RecvAndParsePackets	void FClientNetworkWorker::RecvAndParsePackets()	Recv And Parse Packets 기능을 수행하는 함수입니다.
HandlePacket	void FClientNetworkWorker::HandlePacket(FPacketHeader* Header, uint8* PayloadData, uint16 PayloadSize)	Packet 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
sizeof	PayloadSize < sizeof(int32))	sizeof 기능을 수행하는 함수입니다.

이름	시그니처	설명
sizeof	PayloadSize < sizeof(int32) * 2)	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FAttackActionPacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FDamageApplyPacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FStateChangePacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FStatusUpdatePacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FRespawnPacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FSyncAnimationPacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FRoleChangePacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FSyncGameTimePacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FMapEventPacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FSpawnObjectPacket))	sizeof 기능을 수행하는 함수입니다.
sizeof	PayloadSize < sizeof(FPhaseChangePacket))	sizeof 기능을 수행하는 함수입니다.
SendRawPacket	void FClientNetworkWorker::SendRawPacket(uint16 PacketID, const TArray<uint8>& Payload)	Raw Packet 데이터를 네트워크나 다른 시스템으로 전달합니다.
PushEvent	void FClientNetworkWorker::PushEvent(FNetEvent&& Event)	Push Event 기능을 수행하는 함수입니다.
HandleReconnect	void FClientNetworkWorker::HandleReconnect()	Reconnect 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.

Networking₩ClientNetworking.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Networking₩ClientNetworking.h`
- 파일 목적: 네트워크 연결, 패킷 전송/수신, 이벤트 분배를 담당하는 파일입니다.
- 함수 수(추출): 17
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
TestFuncInput	void ClientNetworking::TestFuncInput(PacketID testPacket)	Test Func Input 기능을 수행하는 함수입니다.
Start	void ClientNetworking::Start(const FString& IP, int32 Port)	Start 기능을 수행하는 함수입니다.
Shutdown	void ClientNetworking::Shutdown()	종료 시점 정리와 리소스 해제를 수행합니다.
PumpEvents	void ClientNetworking::PumpEvents()	Pump Events 기능을 수행하는 함수입니다.
EnqueueSendCommand	void ClientNetworking::EnqueueSendCommand(uint16 PacketID, const TArray<uint8>& Payload)	Enqueue Send Command 기능을 수행하는 함수입니다.
EnqueueSendCommand	void ClientNetworking::EnqueueSendCommand(uint16 PacketID)	Enqueue Send Command 기능을 수행하는 함수입니다.
RequestConnect	void ClientNetworking::RequestConnect(const FString& IP, int32 Port)	Connect 요청 패킷 또는 서버 호출을 전송합니다.
RequestDisconnect	void ClientNetworking::RequestDisconnect()	Disconnect 요청 패킷 또는 서버 호출을 전송합니다.
RequestReconnect	void ClientNetworking::RequestReconnect()	Reconnect 요청 패킷 또는 서버 호출을 전송합니다.
LoginAccess	void ClientNetworking::LoginAccess(FString& ID, FString& PW)	Login Access 기능을 수행하는 함수입니다.
RegisterRequest	void ClientNetworking::RegisterRequest(FString& ID, FString& PW)	Register Request 기능을 수행하는 함수입니다.
CreateRoomRequest	void ClientNetworking::CreateRoomRequest()	Room Request 객체 또는 리소스를 생성합니다.

이름	시그니처	설명
CharacterSelectRequest	void ClientNetworking::CharacterSelectRequest(int32 RoleType)	Character Select Request 기능을 수행하는 함수입니다.
JoinRoomRequest	void ClientNetworking::JoinRoomRequest(int32 RoomID)	Room Request 흐름에 참가하거나 연결을 수립합니다.
ReadyToggleRequest	void ClientNetworking::ReadyToggleRequest()	Ready Toggle Request 기능을 수행하는 함수입니다.
GameStartRequest	void ClientNetworking::GameStartRequest()	Game Start Request 기능을 수행하는 함수입니다.
SendMoveInput	void ClientNetworking::SendMoveInput(const FMovePacket& MoveData)	Move Input 데이터를 네트워크나 다른 시스템으로 전달합니다.

Sub₩HPComponent.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Sub₩HPComponent.h`
- 파일 목적: 체력 관련 공통 계산과 상태 표시 보조를 담당하는 파일입니다.
- 함수 수(추출): 9
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void UHPComponent::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.
ApplyDamage	void UHPComponent::ApplyDamage(int32 DamageAmount)	Damage 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
Heal	void UHPComponent::Heal(int32 HealAmount)	Heal 기능을 수행하는 함수입니다.
ResetHP	void UHPComponent::ResetHP()	Reset HP 기능을 수행하는 함수입니다.
ServerResetHP_Implementation	void UHPComponent::ServerResetHP_Implementation()	Server Reset Implementation 기능을 수행하는 함수입니다.
OnRep_CurrentHP	void UHPComponent::OnRep_CurrentHP(int32 OldHP)	Current HP 복제 값이 변경되었을 때 후속 동기화를 수행합니다.
HandleHPChanged	void UHPComponent::HandleHPChanged(int32 OldHP)	HP Changed 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
ServerApplyDamage_Implementation	void UHPComponent::ServerApplyDamage_Implementation(int32 DamageAmount)	Server Apply Damage Implementation 기능을 수행하는 함수입니다.
ServerHeal_Implementation	void UHPComponent::ServerHeal_Implementation(int32 HealAmount)	Server Heal Implementation 기능을 수행하는 함수입니다.

Sub₩MyCharacter.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩Sub₩MyCharacter.h`
- 파일 목적: 플레이어 캐릭터의 입력, 상태, 역할, 전투, 네트워크 동기화 적용을 담당하는 핵심 파일입니다.
- 함수 수(추출): 80
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
BeginPlay	void AMyCharacter::BeginPlay()	액터가 월드에 배치된 직후 실행되는 시작 로직입니다.

이름	시그니처	설명
Tick	void AMyCharacter::Tick(float DeltaTime)	매 프레임 또는 주기적으로 호출되어 상태를 갱신합니다.
OnRep_CurrentHP	void AMyCharacter::OnRep_CurrentHP()	Current HP 복제 값이 변경되었을 때 후속 동기화를 수행합니다.
OnRep_CharacterState	void AMyCharacter::OnRep_CharacterState()	Character State 복제 값이 변경되었을 때 후속 동기화를 수행합니다.
OnRep_RoleType	void AMyCharacter::OnRep_RoleType()	Role Type 복제 값이 변경되었을 때 후속 동기화를 수행합니다.
OnRep_RoleSkillActive	void AMyCharacter::OnRep_RoleSkillActive()	Role Skill Active 복제 값이 변경되었을 때 후속 동기화를 수행합니다.
UpdateHPText	void AMyCharacter::UpdateHPText()	HP Text 상태나 표시값을 최신 값으로 갱신합니다.
UpdateRoleText	void AMyCharacter::UpdateRoleText()	Role Text 상태나 표시값을 최신 값으로 갱신합니다.
GetRoleSpeedMultiplier	float AMyCharacter::GetRoleSpeedMultiplier(ERecRoleType InType)	Role Speed Multiplier 값을 조회하여 반환합니다.
ApplyRoleStats	void AMyCharacter::ApplyRoleStats()	Role Stats 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
ApplyRoleVisual	void AMyCharacter::ApplyRoleVisual()	Role Visual 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
ApplyRoleSkillState	void AMyCharacter::ApplyRoleSkillState()	Role Skill State 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
RoleTypeToString	FString AMyCharacter::RoleTypeToString(ERecRoleType InType)	Role Type To String 기능을 수행하는 함수입니다.
SetCurrentHP	void AMyCharacter::SetCurrentHP(int32 NewHP)	Current HP 값을 현재 객체에 설정합니다.
SetCharacterState	void AMyCharacter::SetCharacterState(ERecCharacterState NewState)	Character State 값을 현재 객체에 설정합니다.
ApplyCharacterState	void AMyCharacter::ApplyCharacterState()	Character State 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
ApplyDamage	void AMyCharacter::ApplyDamage(int32 DamageAmount)	Damage 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
Heal	void AMyCharacter::Heal(int32 HealAmount)	Heal 기능을 수행하는 함수입니다.
ResetHP	void AMyCharacter::ResetHP()	Reset HP 기능을 수행하는 함수입니다.
ActivateRoleSkill	void AMyCharacter::ActivateRoleSkill()	Activate Role Skill 기능을 수행하는 함수입니다.
EndRoleSkill	void AMyCharacter::EndRoleSkill()	End Role Skill 기능을 수행하는 함수입니다.
ServerApplyDamage_Implementation	void AMyCharacter::ServerApplyDamage_Implementation(int32 DamageAmount)	Server Apply Damage Implementation 기능을 수행하는 함수입니다.
ServerHeal_Implementation	void AMyCharacter::ServerHeal_Implementation(int32 HealAmount)	Server Heal Implementation 기능을 수행하는 함수입니다.
ServerResetHP_Implementation	void AMyCharacter::ServerResetHP_Implementation()	Server Reset Implementation 기능을 수행하는 함수입니다.
ServerCycleRole_Implementation	void AMyCharacter::ServerCycleRole_Implementation()	Server Cycle Role Implementation 기능을 수행하는 함수입니다.
ServerActivateRoleSkill_Implementation	void AMyCharacter::ServerActivateRoleSkill_Implementation()	Server Activate Role Skill Implementation 기능을 수행하는 함수입니다.
TakeDamage	float AMyCharacter::TakeDamage(float DamageAmount, FDamageEvent const& DamageEvent, AController* EventInstigator, AActor* DamageCauser)	Take Damage 기능을 수행하는 함수입니다.
SetupPlayerInputComponent	void AMyCharacter::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)	up Player Input Component 값을 현재 객체에 설정합니다.
TestFunc	void AMyCharacter::TestFunc()	Test Func 기능을 수행하는 함수입니다.

이름	시그니처	설명
Move	void AMyCharacter::Move(const FInputActionValue& Value)	Move 기능을 수행하는 함수입니다.
Look	void AMyCharacter::Look(const FInputActionValue& Value)	Look 기능을 수행하는 함수입니다.
DoMove	void AMyCharacter::DoMove(float Right, float Forward)	Do Move 기능을 수행하는 함수입니다.
DoLook	void AMyCharacter::DoLook(float Yaw, float Pitch)	Do Look 기능을 수행하는 함수입니다.
DoJumpStart	void AMyCharacter::DoJumpStart()	Do Jump Start 기능을 수행하는 함수입니다.
DoJumpEnd	void AMyCharacter::DoJumpEnd()	Do Jump End 기능을 수행하는 함수입니다.
Jump	void AMyCharacter::Jump()	Jump 기능을 수행하는 함수입니다.
SelfDamagePressed	void AMyCharacter::SelfDamagePressed()	Self Damage Pressed 기능을 수행하는 함수입니다.
ActivateRoleSkillPressed	void AMyCharacter::ActivateRoleSkillPressed()	Activate Role Skill Pressed 기능을 수행하는 함수입니다.
UpdateLocalPostProcessEffects	void AMyCharacter::UpdateLocalPostProcessEffects()	Local Post Process Effects 상태나 표시값을 최신 값으로 갱신합니다.
UpdateLowHPEffectState	void AMyCharacter::UpdateLowHPEffectState()	Low HP Effect State 상태나 표시값을 최신 값으로 갱신합니다.
UpdateLowHPPulseEffect	void AMyCharacter::UpdateLowHPPulseEffect(float DeltaTime)	Low HP Pulse Effect 상태나 표시값을 최신 값으로 갱신합니다.
StartDeathCameraShake	void AMyCharacter::StartDeathCameraShake()	Start Death Camera Shake 기능을 수행하는 함수입니다.
UpdateDeathCameraShake	void AMyCharacter::UpdateDeathCameraShake(float DeltaTime)	Death Camera Shake 상태나 표시값을 최신 값으로 갱신합니다.
StopDeathCameraShake	void AMyCharacter::StopDeathCameraShake()	Stop Death Camera Shake 기능을 수행하는 함수입니다.
UpdateDeathCameraPullback	void AMyCharacter::UpdateDeathCameraPullback(float DeltaTime)	Death Camera Pullback 상태나 표시값을 최신 값으로 갱신합니다.
SetDeathVisualsEnabled	void AMyCharacter::SetDeathVisualsEnabled(bool bEnabled)	Death Visuals Enabled 값을 현재 객체에 설정합니다.
SetPlayerControlLocked	void AMyCharacter::SetPlayerControlLocked(bool bLocked)	Player Control Locked 값을 현재 객체에 설정합니다.
UpdateDeathUI	void AMyCharacter::UpdateDeathUI()	Death UI 상태나 표시값을 최신 값으로 갱신합니다.
ShowSpectatingUI	void AMyCharacter::ShowSpectatingUI()	화면 요소 또는 UI를 생성/표시합니다.
HideDeathUI	void AMyCharacter::HideDeathUI()	Hide Death UI 기능을 수행하는 함수입니다.
ShowCorpse	void AMyCharacter::ShowCorpse()	화면 요소 또는 UI를 생성/표시합니다.
HideCorpse	void AMyCharacter::HideCorpse()	Hide Corpse 기능을 수행하는 함수입니다.
EnterDeathWaitingState	void AMyCharacter::EnterDeathWaitingState()	Enter Death Waiting State 기능을 수행하는 함수입니다.
UnlockSpectateInput	void AMyCharacter::UnlockSpectateInput()	Unlock Spectate Input 기능을 수행하는 함수입니다.
EnterSpectateMode	void AMyCharacter::EnterSpectateMode()	Enter Spectate Mode 기능을 수행하는 함수입니다.
ApplyDeadState	void AMyCharacter::ApplyDeadState()	Dead State 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
ApplyAliveState	void AMyCharacter::ApplyAliveState()	Alive State 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
RespawnAtPlayerStart	void AMyCharacter::RespawnAtPlayerStart()	Respawn At Player Start 기능을 수행하는 함수입니다.
SpectateMove	void AMyCharacter::SpectateMove(float Right, float Forward)	Spectate Move 기능을 수행하는 함수입니다.
SpectateUpPressed	void AMyCharacter::SpectateUpPressed()	Spectate Up Pressed 기능을 수행하는 함수입니다.
SpectateUpReleased	void AMyCharacter::SpectateUpReleased()	Spectate Up Released 기능을 수행하는 함수입니다.

이름	시그니처	설명
SpectateDownPressed	void AMyCharacter::SpectateDownPressed()	Spectate Down Pressed 기능을 수행하는 함수입니다.
SpectateDownReleased	void AMyCharacter::SpectateDownReleased()	Spectate Down Released 기능을 수행하는 함수입니다.
SpectateMoveVertical	void AMyCharacter::SpectateMoveVertical(float Axis, float DeltaTime)	Spectate Move Vertical 기능을 수행하는 함수입니다.
ResurrectPressed	void AMyCharacter::ResurrectPressed()	Resurrect Pressed 기능을 수행하는 함수입니다.
CycleRolePressed	void AMyCharacter::CycleRolePressed()	Cycle Role Pressed 기능을 수행하는 함수입니다.
SpectateConfirmPressed	void AMyCharacter::SpectateConfirmPressed()	Spectate Confirm Pressed 기능을 수행하는 함수입니다.
Attack	void AMyCharacter::Attack()	Attack 기능을 수행하는 함수입니다.
PlayAttackMontageFromServer	void AMyCharacter::PlayAttackMontageFromServer(int32 AttackType)	Play Attack Montage From Server 기능을 수행하는 함수입니다.
SetRoleFromNetwork	void AMyCharacter::SetRoleFromNetwork(int32 InRoleType)	Role From Network 값을 현재 객체에 설정합니다.
SetHPFromNetwork	void AMyCharacter::SetHPFromNetwork(int32 InHP)	HP From Network 값을 현재 객체에 설정합니다.
SetStateFromNetwork	void AMyCharacter::SetStateFromNetwork(int32 InState)	State From Network 값을 현재 객체에 설정합니다.
ApplyTransformFromNetwork	void AMyCharacter::ApplyTransformFromNetwork(float X, float Y, float Z, float Yaw)	Transform From Network 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
KnockbackTest	void AMyCharacter::KnockbackTest()	Knockback Test 기능을 수행하는 함수입니다.
ApplyKnockback	void AMyCharacter::ApplyKnockback(AActor* Attacker, float KnockbackStrength)	Knockback 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
OnAttackOverlap	void AMyCharacter::OnAttackOverlap(UPrimitiveComponent* OverlappedComponent, AActor* OtherActor, UPrimitiveComponent* OtherComp, int32 OtherBodyIndex, bool bFromSweep, const FHitResult& SweepResult)	On Attack Overlap 기능을 수행하는 함수입니다.
SetOverlappingItem	void AMyCharacter::SetOverlappingItem(ABaseItem* Item)	Overlapping Item 값을 현재 객체에 설정합니다.
EquipItem	void AMyCharacter::EquipItem()	Equip Item 기능을 수행하는 함수입니다.
DropCurrentItem	void AMyCharacter::DropCurrentItem()	Drop Current Item 기능을 수행하는 함수입니다.
AnimNotify_AttackHit	void AMyCharacter::AnimNotify_AttackHit()	Anim Notify Attack Hit 기능을 수행하는 함수입니다.

UI₩LoginWidget.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩UI₩LoginWidget.h`
- 파일 목적: 로그인, 로비, 방, HUD 등 사용자 인터페이스 흐름을 담당하는 파일입니다.
- 함수 수(추출): 7
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
FocusInitialWidget	void ULoginWidget::FocusInitialWidget()	Focus Initial Widget 기능을 수행하는 함수입니다.
ShowPopup	void ULoginWidget::ShowPopup(const FText& PopupMessage)	화면 요소 또는 UI를 생성/표시합니다.
HidePopup	void ULoginWidget::HidePopup()	Hide Popup 기능을 수행하는 함수입니다.
NativeConstruct	void ULoginWidget::NativeConstruct()	Native Construct 기능을 수행하는 함수입니다.

이름	시그니처	설명
NativeOnMouseButtonDown	FReply ULoginWidget::NativeOnMouseButtonDown(const FGeometry& InGeometry, const FPointerEvent& InMouseEvent)	Native On Mouse Button Down 기능을 수행하는 함수입니다.
OnLoginClicked	void ULoginWidget::OnLoginClicked()	On Login Clicked 기능을 수행하는 함수입니다.
OnRegisterCliecked	void ULoginWidget::OnRegisterCliecked()	On Register Cliecked 기능을 수행하는 함수입니다.

UI₩NetworkInstance.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩UI₩NetworkInstance.h`
- 파일 목적: 로그인, 로비, 방, HUD 등 사용자 인터페이스 흐름을 담당하는 파일입니다.
- 함수 수(추출): 29
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
Init	void UNetworkInstance::Init()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
Shutdown	void UNetworkInstance::Shutdown()	종료 시점 정리와 리소스 해제를 수행합니다.
Tick	void UNetworkInstance::Tick(float DeltaTime)	매 프레임 또는 주기적으로 호출되어 상태를 갱신합니다.
SelectCharacterAndReady	void UNetworkInstance::SelectCharacterAndReady(int32 SelectedRoleType)	Select Character And Ready 기능을 수행하는 함수입니다.
ShowLoginHUD	void UNetworkInstance::ShowLoginHUD()	화면 요소 또는 UI를 생성/표시합니다.
TryLogin	void UNetworkInstance::TryLogin(FString ID, FString PW)	조건을 확인한 뒤 요청 또는 동작을 시도합니다.
TryRegister	void UNetworkInstance::TryRegister(FString ID, FString PW)	조건을 확인한 뒤 요청 또는 동작을 시도합니다.
CreateRoom	void UNetworkInstance::CreateRoom()	Room 객체 또는 리소스를 생성합니다.
JoinRoom	void UNetworkInstance::JoinRoom(int32 RoomID)	Room 흐름에 참가하거나 연결을 수립합니다.
CharacterSelectRequest	void UNetworkInstance::CharacterSelectRequest()	Character Select Request 기능을 수행하는 함수입니다.
RequestReady	void UNetworkInstance::RequestReady()	Ready 요청 패킷 또는 서버 호출을 전송합니다.
RequestGameStart	void UNetworkInstance::RequestGameStart()	Game Start 요청 패킷 또는 서버 호출을 전송합니다.
SendMoveInputToServer	void UNetworkInstance::SendMoveInputToServer(const FMovePacket& MoveData)	Move Input To Server 데이터를 네트워크나 다른 시스템으로 전달합니다.
SendGameplayTestPacket	void UNetworkInstance::SendGameplayTestPacket(Packet ID TestPacket)	Gameplay Test Packet 데이터를 네트워크나 다른 시스템으로 전달합니다.
HandleLoginResult	void UNetworkInstance::HandleLoginResult(bool bSuccess)	Login Result 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleRegisterResult	void UNetworkInstance::HandleRegisterResult(bool bSuccess)	Register Result 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleRoomListUpdate	void UNetworkInstance::HandleRoomListUpdate(const TArray<FRoomInfoData>& RoomList)	Room List Update 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleRoomEnterResult	void UNetworkInstance::HandleRoomEnterResult(bool bSuccess, const TArray<FRoomMemberInfo>& playerList)	Room Enter Result 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleGameStart	void UNetworkInstance::HandleGameStart()	Game Start 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.

이름	시그니처	설명
HandleConnected	void UNetworkInstance::HandleConnected()	Connected 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleConnectFailed	void UNetworkInstance::HandleConnectFailed()	Connect Failed 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleDisconnected	void UNetworkInstance::HandleDisconnected()	Disconnected 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleSnapshotReceived	void UNetworkInstance::HandleSnapshotReceived(const TArray<GameData>& SnapshotList)	Snapshot Received 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleAttackActionReceived	void UNetworkInstance::HandleAttackActionReceived(const FAttackActionPacket& Packet)	Attack Action Received 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleDamageApplied	void UNetworkInstance::HandleDamageApplied(const FDamageApplyPacket& Packet)	Damage Applied 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleStatusUpdated	void UNetworkInstance::HandleStatusUpdated(const FStatusUpdatePacket& Packet)	Status Updated 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleStateChanged	void UNetworkInstance::HandleStateChanged(const FStateChangePacket& Packet)	State Changed 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleRespawned	void UNetworkInstance::HandleRespawned(const FRespawnPacket& Packet)	Respawned 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
EnsureRemoteCharacter	AMyCharacter* UNetworkInstance::EnsureRemoteCharacter(const GameData& Data)	Remote Character 객체가 없으면 생성하고, 있으면 재사용합니다.

UI₩RobbyWidget.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩UI₩RobbyWidget.h`
- 파일 목적: 로그인, 로비, 방, HUD 등 사용자 인터페이스 흐름을 담당하는 파일입니다.
- 함수 수(추출): 4
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
NativeConstruct	void URobbyWidget::NativeConstruct()	Native Construct 기능을 수행하는 함수입니다.
ReloadRoomList	void URobbyWidget::ReloadRoomList(const TArray<FRoomInfoData>& RoomList)	Reload Room List 기능을 수행하는 함수입니다.
OnCreateButtonClicked	void URobbyWidget::OnCreateButtonClicked()	On Create Button Clicked 기능을 수행하는 함수입니다.
OnStartButtonClicked	void URobbyWidget::OnStartButtonClicked()	On Start Button Clicked 기능을 수행하는 함수입니다.

UI₩RoomEnterWidget.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩UI₩RoomEnterWidget.h`
- 파일 목적: 로그인, 로비, 방, HUD 등 사용자 인터페이스 흐름을 담당하는 파일입니다.
- 함수 수(추출): 4
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
NativeConstruct	void URoomEnterWidget::NativeConstruct()	Native Construct 기능을 수행하는 함수입니다.
InitRoomInfo	void URoomEnterWidget::InitRoomInfo(int32 s_RoomID, const FString& s_RoomName, int32 CurrentPlayers, int32 MaxPlayers)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
UpdatePlayerCount	void URoomEnterWidget::UpdatePlayerCount(int32 CurrentPlayers, int32 MaxPlayers)	Player Count 상태나 표시값을 최신 값으로 갱신합니다.
OnJoinButtonClicked	void URoomEnterWidget::OnJoinButtonClicked()	On Join Button Clicked 기능을 수행하는 함수입니다.

UI₩RoomMemberWidget.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩UI₩RoomMemberWidget.h`
- 파일 목적: 로그인, 로비, 방, HUD 등 사용자 인터페이스 흐름을 담당하는 파일입니다.
- 함수 수(추출): 8
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
NativeConstruct	void URoomMemberWidget::NativeConstruct()	Native Construct 기능을 수행하는 함수입니다.
NativeDestruct	void URoomMemberWidget::NativeDestruct()	Native Destruct 기능을 수행하는 함수입니다.
SetEmptyMember	void URoomMemberWidget::SetEmptyMember()	Empty Member 값을 현재 객체에 설정합니다.
SetMemberInfo	void URoomMemberWidget::SetMemberInfo(const FString& playerName, bool bIsReady)	Member Info 값을 현재 객체에 설정합니다.
OnStrikerClicked	void URoomMemberWidget::OnStrikerClicked()	On Striker Clicked 기능을 수행하는 함수입니다.
OnGuardianClicked	void URoomMemberWidget::OnGuardianClicked()	On Guardian Clicked 기능을 수행하는 함수입니다.
OnManipulatorClicked	void URoomMemberWidget::OnManipulatorClicked()	On Manipulator Clicked 기능을 수행하는 함수입니다.
HandleRoleSelected	void URoomMemberWidget::HandleRoleSelected(int32 InRoleType)	Role Selected 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.

UI₩RoomWidget.h

- 절대 경로: `D:₩Download₩Claim_Of_Core_Project-MainClient₩Claim_Of_Core_Project-MainClient₩MainClient_Codes₩Claim_Of_Core₩Source₩Claim_Of_Core₩UI₩RoomWidget.h`
- 파일 목적: 로그인, 로비, 방, HUD 등 사용자 인터페이스 흐름을 담당하는 파일입니다.
- 함수 수(추출): 3
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
NativeConstruct	void URoomWidget::NativeConstruct()	Native Construct 기능을 수행하는 함수입니다.
UpdateMemberList	void URoomWidget::UpdateMemberList(const TArray<FRoomMemberInfo>& PlayerList)	Member List 상태나 표시값을 최신 값으로 갱신합니다.
OnStartButtonClicked	void URoomWidget::OnStartButtonClicked()	On Start Button Clicked 기능을 수행하는 함수입니다.

서버

Database.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩Database.h`
- 파일 목적: DB 연결, 계정 조회/생성, 영속 데이터 접근을 담당하는 파일입니다.
- 함수 수(추출): 8
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
Connect	bool DBHelper::Connect()	Connect 기능을 수행하는 함수입니다.
Disconnect	void DBHelper::Disconnect()	Disconnect 기능을 수행하는 함수입니다.
PrintError	void DBHelper::PrintError(SQLHANDLE handle, SQLSMALLINT type)	Print Error 기능을 수행하는 함수입니다.
CheckLogin	bool DBHelper::CheckLogin(const std::wstring& userID, const std::wstring& userPW, int& playerUID)	Check Login 기능을 수행하는 함수입니다.
IsValidAccountString	bool DBHelper::IsValidAccountString(const std::wstring& inputStr)	Is Valid Account String 기능을 수행하는 함수입니다.
IsValidUserID	bool DBHelper::IsValidUserID(const std::wstring& inputID)	Is Valid User ID 기능을 수행하는 함수입니다.
IsValidUserPW	bool DBHelper::IsValidUserPW(const std::wstring& inputPW)	Is Valid User PW 기능을 수행하는 함수입니다.
CreateAccount	bool DBHelper::CreateAccount(const std::wstring& userID, const std::wstring& userPW)	Account 객체 또는 리소스를 생성합니다.

GameLogic.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩GameLogic.h`
- 파일 목적: 서버 측 게임 규칙, 라운드 상태, 이동/전투/판정 계산을 담당하는 파일입니다.
- 주요 타입: `ObjectData, ItemData, ObjectManager, IcicleData, IceFloorTileData, DebrisSpawnConfig, CollisionBox`
- 함수 수(추출): 45
- 멤버 변수 수(추출): 19

멤버 변수

이름	타입	설명
ObjectID	int	ObjectID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
ObjectType	e_ObjectType	ObjectType 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
z	float x, y,	z 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
ObjectID	int	ObjectID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
ObjectType	e_ObjectType	ObjectType 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
z	float x, y,	z 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
Data	ObjectData	Data 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
icicleID	int	icicleID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
icicleState	EIcicleState	icicleState 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

이름	타입	설명
warningTimer	float	warningTimer 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
warningDuration	float	warningDuration 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
damage	float	damage 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
tileID	int	tileID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
tileState	EIceFloorState	tileState 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
breakTimer	float	기능 토글 또는 현재 상태를 나타내는 불리언 플래그입니다.
breakDelay	float	기능 토글 또는 현재 상태를 나타내는 불리언 플래그입니다.
max_Interval	float min_Interval,	max_Interval 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
max_Count	int min_Count,	max_Count 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
MinX	float	MinX 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

함수

이름	시그니처	설명
ObjectManager	ObjectManager()	Object Manager 기능을 수행하는 함수입니다.
Collision_Check	void Collision_Check()	Collision Check 기능을 수행하는 함수입니다.
Damage_Calculation	void Damage_Calculation()	Damage Calculation 기능을 수행하는 함수입니다.
Update	void GameLogic::Update(float deltaTime)	state 상태나 표시값을 최신 값으로 갱신합니다.
StartGameReady	void GameLogic::StartGameReady()	Start Game Ready 기능을 수행하는 함수입니다.
StartGameRound	void GameLogic::StartGameRound()	Start Game Round 기능을 수행하는 함수입니다.
EndGameRound	void GameLogic::EndGameRound()	End Game Round 기능을 수행하는 함수입니다.
CountdownTick	void GameLogic::CountdownTick()	Countdown Tick 기능을 수행하는 함수입니다.
HandlePhaseChanged	void GameLogic::HandlePhaseChanged(int newPhase)	Phase Changed 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
BroadcastGameTime	void GameLogic::BroadcastGameTime(float currentTime)	Broadcast Game Time 기능을 수행하는 함수입니다.
BroadcastDamageApply	void GameLogic::BroadcastDamageApply(int targetID, int damage, int remainHP)	Broadcast Damage Apply 기능을 수행하는 함수입니다.
BroadcastStatusUpdate	void GameLogic::BroadcastStatusUpdate(int targetID, int currentHP)	Broadcast Status Update 기능을 수행하는 함수입니다.
BroadcastStateChange	void GameLogic::BroadcastStateChange(int targetID, int newState)	Broadcast State Change 기능을 수행하는 함수입니다.
BroadcastRespawn	void GameLogic::BroadcastRespawn(int targetID, float x, float y, float z, int hp)	Broadcast Respawn 기능을 수행하는 함수입니다.
BroadcastRoleChange	void GameLogic::BroadcastRoleChange(int targetID, int newRoleType)	Broadcast Role Change 기능을 수행하는 함수입니다.
ApplyDamage	void GameLogic::ApplyDamage(int sessionID, int damageAmount)	Damage 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
Heal	void GameLogic::Heal(int sessionID, int healAmount)	Heal 기능을 수행하는 함수입니다.
ResetHP	void GameLogic::ResetHP(int sessionID)	Reset HP 기능을 수행하는 함수입니다.
SetCurrentHP	void GameLogic::SetCurrentHP(int sessionID, int newHP)	Current HP 값을 현재 객체에 설정합니다.

이름	시그니처	설명
SetCharacterState	void GameLogic::SetCharacterState(int sessionID, int newState)	Character State 값을 현재 객체에 설정합니다.
HandleDeath	void GameLogic::HandleDeath(int sessionID)	Death 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
HandleRespawn	void GameLogic::HandleRespawn(int sessionID)	Respawn 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
SetRole	void GameLogic::SetRole(int sessionID)	Role 값을 현재 객체에 설정합니다.
ApplyRoleStats	void GameLogic::ApplyRoleStats(int sessionID)	Role Stats 계산 결과를 컴포넌트나 현재 객체 상태에 반영합니다.
GetRoleSpeedMultiplier	float GameLogic::GetRoleSpeedMultiplier(int roleType)	Role Speed Multiplier 값을 조회하여 반환합니다.
HandleAttackInput	void GameLogic::HandleAttackInput(int sessionID)	Attack Input 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
BroadcastAttackAction	void GameLogic::BroadcastAttackAction(int attackerUID, int attackType)	Broadcast Attack Action 기능을 수행하는 함수입니다.
UpdateBuildingMap	void GameLogic::UpdateBuildingMap(float deltaTime)	Building Map 상태나 표시값을 최신 값으로 갱신합니다.
UpdateDebrisSpawner	void GameLogic::UpdateDebrisSpawner(float deltaTime)	Debris Spawner 상태나 표시값을 최신 값으로 갱신합니다.
SpawnDebrisByPhase	void GameLogic::SpawnDebrisByPhase()	Spawn Debris By Phase 기능을 수행하는 함수입니다.
TriggerBuildingPhase2	void GameLogic::TriggerBuildingPhase2()	Trigger Building Phase 2 기능을 수행하는 함수입니다.
TriggerBuildingPhase3	void GameLogic::TriggerBuildingPhase3()	Trigger Building Phase 3 기능을 수행하는 함수입니다.
StartIceMap	void GameLogic::StartIceMap()	Start Ice Map 기능을 수행하는 함수입니다.
UpdateIceCaveMap	void GameLogic::UpdateIceCaveMap(float deltaTime)	Ice Cave Map 상태나 표시값을 최신 값으로 갱신합니다.
EnterIcePhase2	void GameLogic::EnterIcePhase2()	Enter Ice Phase 2 기능을 수행하는 함수입니다.
EnterIcePhase3	void GameLogic::EnterIcePhase3()	Enter Ice Phase 3 기능을 수행하는 함수입니다.
TriggerRandomIcicle	void GameLogic::TriggerRandomIcicle()	Trigger Random Icicle 기능을 수행하는 함수입니다.
StartIcicleWarning	void GameLogic::StartIcicleWarning(int icicleID)	Start Icicle Warning 기능을 수행하는 함수입니다.
StartIcicleFalling	void GameLogic::StartIcicleFalling(int icicleID)	Start Icicle Falling 기능을 수행하는 함수입니다.
BreakIcicle	void GameLogic::BreakIcicle(int icicleID)	Break Icicle 기능을 수행하는 함수입니다.
TriggerRandomFloorTile	void GameLogic::TriggerRandomFloorTile()	Trigger Random Floor Tile 기능을 수행하는 함수입니다.
SetFloorCracked	void GameLogic::SetFloorCracked(int tileID)	Floor Cracked 값을 현재 객체에 설정합니다.
StartFloorBreaking	void GameLogic::StartFloorBreaking(int tileID)	Start Floor Breaking 기능을 수행하는 함수입니다.
BreakFloor	void GameLogic::BreakFloor(int tileID)	Break Floor 기능을 수행하는 함수입니다.
UpdatePlayerMovement	void GameLogic::UpdatePlayerMovement(Session* player, float deltaTime)	Player Movement 상태나 표시값을 최신 값으로 갱신합니다.

Packet.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩Packet.h`
- 파일 목적: 패킷 ID, 직렬화 포맷, 송수신 데이터 구조를 정의하는 파일입니다.
- 주요 타입: `PacketHeader, GameData, ErrorCodePacket, LoginPacket, RoomPacket, RoomMemberPacket, GameDataPacket, MapEventPacket, SpawnObjectPacket, PhaseChangePacket, DamageApplyPacket, StatusUpdatePacket, StateChangePacket, RespawnPacket, RoleChangePacket, MovePacket, JumpPakcet, AttackPacket, ItemPacket`
- 함수 수(추출): 0
- 멤버 변수 수(추출): 34

멤버 변수

이름	타입	설명
packet_size	unsigned short	packet_size 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
packet_ID	unsigned short	packet_ID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
userUID	int	세션/플레이어 식별자와 관련된 상태입니다.
isConnected	bool	isConnected 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
maxHP	int	체력 관련 현재값/최대값을 저장합니다.
currentHP	int	체력 관련 현재값/최대값을 저장합니다.
x	float	x 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
rotate	float	rotate 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
baseWalkSpeed	float	기능 토글 또는 현재 상태를 나타내는 불리언 플래그입니다.
animationNum	int	animationNum 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
ErrorCode	int16_t	ErrorCode 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
userID[20]	WCHAR	userID[20] 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
userPW[20]	WCHAR	userPW[20] 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
roomID	int	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
RoomName[20]	WCHAR	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
CurrentPlayers	int	CurrentPlayers 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
PlayerName[20]	WCHAR	PlayerName[20] 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
isReady	bool	isReady 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
isHost	bool	isHost 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
Session_ID	int	세션/플레이어 식별자와 관련된 상태입니다.
data	GameData	data 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
targetID	int32_t	targetID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
damage	int32_t	damage 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
remainHP	int32_t	remainHP 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
targetID	int32_t	targetID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
currentHP	int32_t	체력 관련 현재값/최대값을 저장합니다.
targetID	int32_t	targetID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
targetID	int32_t	targetID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
z	float x, y,	z 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
hp	int32_t	hp 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

이름	타입	설명
targetID	int32_t	targetID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
z	float x, y,	z 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
yaw	float	yaw 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
velocityY	float velocityX,	velocityY 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

PacketProcessor.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩PacketProcessor.h`
- 파일 목적: 수신 패킷을 해석하고 각 기능 처리기로 분배하는 파일입니다.
- 함수 수(추출): 16
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
GameLogicHelper	GameLogic* PacketProcessor::GameLogicHelper(IOCPServer* server, Session* session)	Game Logic Helper 기능을 수행하는 함수입니다.
InitHandler	void PacketProcessor::InitHandler()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
Process	void PacketProcessor::Process(IOCPServer* server, Session* session, int packetID, std::vector<char>& data)	입력 데이터를 해석하고 적절한 처리 흐름으로 연결합니다.
Handle_LoginReq	void PacketProcessor::Handle_LoginReq(IOCPServer* server, Session* session, PacketReader& reader)	Login Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_RegisterReq	void PacketProcessor::Handle_RegisterReq(IOCPServer* server, Session* session, PacketReader& reader)	Register Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Room_CreateReq	void PacketProcessor::Handle_Room_CreateReq(IOCPServer* server, Session* session, PacketReader&)	Room Create Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Room_JoinReq	void PacketProcessor::Handle_Room_JoinReq(IOCPServer* server, Session* session, PacketReader& reader)	Room Join Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Room_RemoveReq	void PacketProcessor::Handle_Room_RemoveReq(IOCPServer* server, Session* session, PacketReader& reader)	Room Remove Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Game_StartReq	void PacketProcessor::Handle_Game_StartReq(IOCPServer* server, Session* session, PacketReader& reader)	Game Start Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Character_SelectReq	void PacketProcessor::Handle_Character_SelectReq(IOCPServer* server, Session* session, PacketReader& reader)	Character Select Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Game_ReadyReq	void PacketProcessor::Handle_Game_ReadyReq(IOCPServer* server, Session* session, PacketReader& reader)	Game Ready Req 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Move_KeyInput	bool PacketProcessor::Handle_Move_KeyInput(IOCPServer* server, Session* session, PacketReader& reader)	Move Key Input 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_Jump_KeyInput	void PacketProcessor::Handle_Jump_KeyInput(IOCPServer* server, Session* session, PacketReader& reader)	Jump Key Input 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.

이름	시그니처	설명
Handle_Attack_KeyInput	void PacketProcessor::Handle_Attack_KeyInput(IOCPServer* server, Session* session, PacketReader& reader)	Attack Key Input 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_ItemPickup_KeyInput	void PacketProcessor::Handle_ItemPickup_KeyInput(IOCPServer* server, Session* session, PacketReader& reader)	Item Pickup Key Input 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.
Handle_ItemDrop_KeyInput	void PacketProcessor::Handle_ItemDrop_KeyInput(IOCPServer* server, Session* session, PacketReader& reader)	Item Drop Key Input 관련 이벤트나 패킷을 받아 후속 처리를 수행합니다.

RoomManager.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩RoomManager.h`
- 파일 목적: 방 생성/참가/시작/브로드캐스트와 룸 상태 관리를 담당하는 파일입니다.
- 주요 타입: `Room`
- 함수 수(추출): 47
- 멤버 변수 수(추출): 12

멤버 변수

이름	타입	설명
m_roomID	int	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
m_OwnerSessionID	int	세션/플레이어 식별자와 관련된 상태입니다.
UsedSlots[MaxMember]	bool	UsedSlots[MaxMember] 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_Server	IOCPServer*	m_Server 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_GameLogic	GameLogic*	m_GameLogic 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_State	ERoomState	m_State 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_Members	std::vector<Session*>	동일한 종류의 데이터를 순서대로 저장하는 컨테이너입니다.
m_RoomLock	std::mutex	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
selectedMapType	int	맵, 레벨, 라운드 흐름과 관련된 참조 또는 상태값입니다.
gameTimer	float	gameTimer 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_GameObjects	std::unordered_map<int, ObjectData>	m_GameObjects 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_ItemObjects	std::unordered_map<int, ItemData>	m_ItemObjects 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

함수

이름	시그니처	설명
clear	m_Members.clear()	clear 기능을 수행하는 함수입니다.
InitRoom	void InitRoom(int id, Session* firstMember, IOCPServer* server)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
InitGameLogic	void InitGameLogic(GameLogic* logic)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
InitCharacter	void InitCharacter(Session* member, GameLogic* logic)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.

이름	시그니처	설명
IsAllReady	bool IsAllReady()	Is All Ready 기능을 수행하는 함수입니다.
addMember	bool addMember(Session* member)	add Member 기능을 수행하는 함수입니다.
AllocateRoomSlot	int AllocateRoomSlot()	Allocate Room Slot 기능을 수행하는 함수입니다.
RemoveMember	void RemoveMember(int clientID)	Remove Member 기능을 수행하는 함수입니다.
TransferOwnership	void TransferOwnership()	Transfer Ownership 기능을 수행하는 함수입니다.
TeamCalculateBySlot	int TeamCalculateBySlot(int roomSlot) const	Team Calculate By Slot 기능을 수행하는 함수입니다.
SelectStage	void SelectStage(int stageNum)	Select Stage 기능을 수행하는 함수입니다.
LoadStage	void LoadStage()	Load Stage 기능을 수행하는 함수입니다.
GetRespawnLocation	Vector3 GetRespawnLocation(int slot)	Respawn Location 값을 조회하여 반환합니다.
GetSpawnYawBySlot	float GetSpawnYawBySlot(int slot) const	Spawn Yaw By Slot 값을 조회하여 반환합니다.
MatchMaking	void MatchMaking()	Match Making 기능을 수행하는 함수입니다.
UpdateGameLogic	void UpdateGameLogic(float deltaTime)	Game Logic 상태나 표시값을 최신 값으로 갱신합니다.
BroadcastGameDatas	bool BroadcastGameDatas()	Broadcast Game Datas 기능을 수행하는 함수입니다.
GetCurrentMemberCount	int GetCurrentMemberCount()	Current Member Count 값을 조회하여 반환합니다.
GetMemberInfoList	std::vector<RoomMemberPacket> GetMemberInfoList()	Member Info List 값을 조회하여 반환합니다.
BroadcastToMembers	void BroadcastToMembers(int packetID, const char* data, int len, int excludeSessionID = -1)	Broadcast To Members 기능을 수행하는 함수입니다.
InitRoom	void Room::InitRoom(int id, Session* firstMember, IOCPServer* server)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
InitGameLogic	void Room::InitGameLogic(GameLogic* logic)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
InitCharacter	void Room::InitCharacter(Session* member, GameLogic* logic)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
IsAllReady	bool Room::IsAllReady()	Is All Ready 기능을 수행하는 함수입니다.
addMember	bool Room::addMember(Session* member)	add Member 기능을 수행하는 함수입니다.
AllocateRoomSlot	int Room::AllocateRoomSlot()	Allocate Room Slot 기능을 수행하는 함수입니다.
RemoveMember	void Room::RemoveMember(int clientID)	Remove Member 기능을 수행하는 함수입니다.
TransferOwnership	void Room::TransferOwnership()	Transfer Ownership 기능을 수행하는 함수입니다.
SelectStage	void Room::SelectStage(int stageNum)	Select Stage 기능을 수행하는 함수입니다.
LoadStage	void Room::LoadStage()	Load Stage 기능을 수행하는 함수입니다.
GetRespawnLocation	Vector3 Room::GetRespawnLocation(int slot)	Respawn Location 값을 조회하여 반환합니다.
MatchMaking	void Room::MatchMaking()	Match Making 기능을 수행하는 함수입니다.
UpdateGameLogic	void Room::UpdateGameLogic(float deltaTime)	Game Logic 상태나 표시값을 최신 값으로 갱신합니다.
BroadcastGameDatas	bool Room::BroadcastGameDatas()	Broadcast Game Datas 기능을 수행하는 함수입니다.
GetCurrentMemberCount	int Room::GetCurrentMemberCount()	Current Member Count 값을 조회하여 반환합니다.
GetMemberInfoList	std::vector<RoomMemberPacket> Room::GetMemberInfoList()	Member Info List 값을 조회하여 반환합니다.
BroadcastToMembers	void Room::BroadcastToMembers(int packetID, const char* data, int len, int excludeSessionID)	Broadcast To Members 기능을 수행하는 함수입니다.
CreateRoom	bool RoomManager::CreateRoom(Session* client)	Room 객체 또는 리소스를 생성합니다.
DestroyRoom	void RoomManager::DestroyRoom(int roomID)	Destroy Room 기능을 수행하는 함수입니다.
LeaveRoom	void RoomManager::LeaveRoom(Session* client)	Leave Room 기능을 수행하는 함수입니다.
LeaveRoomByDisconnect	void RoomManager::LeaveRoomByDisconnect(int sessionID, int roomID)	Leave Room By Disconnect 기능을 수행하는 함수입니다.

이름	시그니처	설명
JoinRoom	bool RoomManager::JoinRoom(Session* client, int roomID)	Room 흐름에 참가하거나 연결을 수립합니다.
GameStart	void RoomManager::GameStart(Session* client)	Game Start 기능을 수행하는 함수입니다.
UpdateRooms	void RoomManager::UpdateRooms(float deltaTime)	Rooms 상태나 표시값을 최신 값으로 갱신합니다.
GetRoomList	std::vector<RoomPacket> RoomManager::GetRoomList()	Room List 값을 조회하여 반환합니다.
GetRoom	Room* RoomManager::GetRoom(int roomID)	Room 값을 조회하여 반환합니다.
assignRoomID	int RoomManager::assignRoomID()	assign Room ID 기능을 수행하는 함수입니다.

SessionManager.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩SessionManager.h`
- 파일 목적: 세션 컨테이너와 세션 등록/제거 정책을 담당하는 파일입니다.
- 함수 수(추출): 5
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
Init	void SessionManager::Init()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
AcceptNewClient	Session* SessionManager::AcceptNewClient(SOCKET clientSocket)	Accept New Client 기능을 수행하는 함수입니다.
DisconnectClient	void SessionManager::DisconnectClient(int clientID)	Disconnect Client 기능을 수행하는 함수입니다.
SetState	void SessionManager::SetState(int sessionID, ESessionState state)	State 값을 현재 객체에 설정합니다.
GetSession	Session* SessionManager::GetSession(int sessionID)	Session 값을 조회하여 반환합니다.

logger.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩logger.h`
- 파일 목적: 서버 로그 출력 매크로와 기록 유틸리티를 제공합니다.
- 함수 수(추출): 0
- 멤버 변수 수(추출): 0

logging.hpp

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩logging.hpp`
- 파일 목적: 서버 런타임 기능을 보조하는 파일입니다.
- 주요 타입: `logger, std_out_logger, file_logger, logger_factory`
- 함수 수(추출): 41
- 멤버 변수 수(추출): 19

멤버 변수

이름	타입	설명
lock	std::mutex	lock 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
output	std::string	output 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

이름	타입	설명
lk{lock}	//std::lock_guard<std::mutex>	lk{lock} 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
message	std::cout <<	message 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
levels	const std::unordered_map<log_level, std::string, enum_hasher>	맵, 레벨, 라운드 흐름과 관련된 참조 또는 상태값입니다.
output	std::string	output 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
message	file <<	message 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
e	throw	e 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
lt{}	std::tm	lt{} 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
buf[16]	char	기능 토글 또는 현재 상태를 나타내는 불리언 플래그입니다.
lt.tm_mday)	lt.tm_year + 1900, lt.tm_mon + 1,	lt.tm_mday) 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
".log"	return file_name + "_" + date +	".log" 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
std::ofstream::app)	std::ofstream::out ₩	std::ofstream::app) 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
file_name	std::string	file_name 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
current_date	std::string	current_date 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
file	std::ofstream	file 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
reopen_interval	std::chrono::seconds	reopen_interval 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
last_reopen	std::chrono::system_clock::time_point	last_reopen 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
creators	std::unordered_map<std::string, logger_creator>	creators 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

함수

이름	시그니처	설명
logger	logger() = delete	logger 기능을 수행하는 함수입니다.
logger	logger(const logging_config_t&) {}	logger 기능을 수행하는 함수입니다.
~logger	virtual ~logger() {}	logger 기능을 수행하는 함수입니다.
log	virtual void log(const std::string&, const log_level) {}	log 기능을 수행하는 함수입니다.
log	virtual void log(const std::string&) {}	log 기능을 수행하는 함수입니다.
std_out_logger	std_out_logger() = delete	std out logger 기능을 수행하는 함수입니다.
reserve	output.reserve(message.length() + 64)	reserve 기능을 수행하는 함수입니다.
append	output.append(timestamp())	append 기능을 수행하는 함수입니다.
append	output.append(levels.find(level)->second)	append 기능을 수행하는 함수입니다.
append	output.append(message)	append 기능을 수행하는 함수입니다.
push_back	output.push_back('₩n')	push back 기능을 수행하는 함수입니다.
log	log(output)	log 기능을 수행하는 함수입니다.

이름	시그니처	설명
flush	std::cout.flush()	flush 기능을 수행하는 함수입니다.
file_logger	file_logger() = delete	file logger 기능을 수행하는 함수입니다.
find	auto name = config.find("file_name")	find 기능을 수행하는 함수입니다.
runtime_error	throw std::runtime_error("No output file provided to file logger")	runtime error 기능을 수행하는 함수입니다.
seconds	reopen_interval = std::chrono::seconds(300)	seconds 기능을 수행하는 함수입니다.
find	auto interval = config.find("reopen_interval")	find 기능을 수행하는 함수입니다.
seconds	reopen_interval = std::chrono::seconds(std::stoul(interval->second))	seconds 기능을 수행하는 함수입니다.
runtime_error	throw std::runtime_error(interval->second + " is not a valid reopen interval")	runtime error 기능을 수행하는 함수입니다.
reopen	//reopen()	reopen 기능을 수행하는 함수입니다.
today_string	current_date = today_string()	today string 기능을 수행하는 함수입니다.
open_file	open_file()	open file 기능을 수행하는 함수입니다.
append	output.append(uncolored.find(level)->second)	append 기능을 수행하는 함수입니다.
lock	lock.lock()	lock 기능을 수행하는 함수입니다.
flush	file.flush()	flush 기능을 수행하는 함수입니다.
unlock	lock.unlock()	unlock 기능을 수행하는 함수입니다.
reopen	reopen()	reopen 기능을 수행하는 함수입니다.
now	auto now = std::chrono::system_clock::now()	now 기능을 수행하는 함수입니다.
open	file.open(file_name, std::ofstream::out ₩ | std::ofstream::app)	open 기능을 수행하는 함수입니다.
now	last_reopen = std::chrono::system_clock::now()	now 기능을 수행하는 함수입니다.
today_string	std::string today = today_string()	today string 기능을 수행하는 함수입니다.
time	std::time_t tt = std::time(nullptr)	time 기능을 수행하는 함수입니다.
localtime_s	localtime_s(&lt, &tt)	localtime s 기능을 수행하는 함수입니다.
emplace	creators.emplace("", [](const logging_config_t& config)->logger* {return new logger(config); })	emplace 기능을 수행하는 함수입니다.
emplace	creators.emplace("std_out", [](const logging_config_t& config)->logger* {return new std_out_logger(config); })	emplace 기능을 수행하는 함수입니다.
emplace	creators.emplace("file", [](const logging_config_t& config)->logger* {return new file_logger(config); })	emplace 기능을 수행하는 함수입니다.
find	auto type = config.find("type")	find 기능을 수행하는 함수입니다.
runtime_error	throw std::runtime_error("Logging factory configuration requires a type of logger")	runtime error 기능을 수행하는 함수입니다.
find	auto found = creators.find(type->second)	find 기능을 수행하는 함수입니다.
runtime_error	throw std::runtime_error("Couldn't produce logger for type: " + type->second)	runtime error 기능을 수행하는 함수입니다.

main.cpp

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩main.cpp`
- 파일 목적: 서버 실행 진입점과 부트스트랩을 담당하는 파일입니다.
- 함수 수(추출): 2
- 멤버 변수 수(추출): 0

함수

이름	시그니처	설명
Test	void Test(IOCPServer* server)	Test 기능을 수행하는 함수입니다.
main	int main()	main 기능을 수행하는 함수입니다.

server.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩server.h`
- 파일 목적: IOCP 서버 초기화, 소켓 처리, 세션 생명주기를 담당하는 파일입니다.
- 주요 타입: `IOCPServer`
- 함수 수(추출): 45
- 멤버 변수 수(추출): 16

멤버 변수

이름	타입	설명
m_hIOCP	HANDLE	m_hIOCP 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_ListenSocket	SOCKET	m_ListenSocket 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_DB	DBHelper*	m_DB 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_DBLoginQueue	std::queue<DBData>	m_DBLoginQueue 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_DBMutex	std::mutex	m_DBMutex 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_DBControler	std::condition_variable	역할 타입, 역할 텍스트, 역할 스킬 상태와 관련된 멤버입니다.
m_PacketProcessor	PacketProcessor	m_PacketProcessor 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_SessionManager	SessionManager*	세션/플레이어 식별자와 관련된 상태입니다.
m_RoomManager	RoomManager*	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
m_WorkerThreads	std::vector<std::thread>	동일한 종류의 데이터를 순서대로 저장하는 컨테이너입니다.
m_AcceptThread	std::thread	m_AcceptThread 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_DBConnectThread	std::thread	m_DBConnectThread 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
m_Sessions	//std::vector<Session*>	세션/플레이어 식별자와 관련된 상태입니다.
m_SessionLock	//std::mutex	세션/플레이어 식별자와 관련된 상태입니다.
m_MaxSessionCount	int	세션/플레이어 식별자와 관련된 상태입니다.
m_IsRunning	bool	m_IsRunning 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

함수

이름	시그니처	설명
IOCPServer	IOCPServer()	IOCP Server 기능을 수행하는 함수입니다.
~IOCPServer	~IOCPServer()	IOCP Server 기능을 수행하는 함수입니다.
InitLogger	static void InitLogger()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
Init	bool Init(int port, int maxSessionCount)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
Start	void Start()	Start 기능을 수행하는 함수입니다.
SendPacket	void SendPacket(int sessionIndex, int packetID, const char* data, int len)	Packet 데이터를 네트워크나 다른 시스템으로 전달합니다.
GameFrameProtocol	void GameFrameProtocol(float deltaTime)	Game Frame Protocol 기능을 수행하는 함수입니다.
PushDBTask	void PushDBTask(DBData data)	Push DB Task 기능을 수행하는 함수입니다.
CreateRoomTry	void CreateRoomTry(Session* Client)	Room Try 객체 또는 리소스를 생성합니다.
JoinRoomTry	void JoinRoomTry(Session* Client, int roomID)	Room Try 흐름에 참가하거나 연결을 수립합니다.
GameStartTry	void GameStartTry(Session* Client)	Game Start Try 기능을 수행하는 함수입니다.
BroadcastRoomList	void BroadcastRoomList()	Broadcast Room List 기능을 수행하는 함수입니다.
BroadcastRoomInfo	void BroadcastRoomInfo(Session* client)	Broadcast Room Info 기능을 수행하는 함수입니다.
TestPacketProcessor	void TestPacketProcessor()	Test Packet Processor 기능을 수행하는 함수입니다.
WorkerThread	void WorkerThread()	Worker Thread 기능을 수행하는 함수입니다.
AcceptThread	void AcceptThread()	Accept Thread 기능을 수행하는 함수입니다.
OnConnect	void OnConnect(Session* newSession, SOCKADDR_IN clientAddr)	On Connect 기능을 수행하는 함수입니다.
OnDisconnect	void OnDisconnect(int sessionIndex)	On Disconnect 기능을 수행하는 함수입니다.
OnRecv	void OnRecv(int sessionIndex, DWORD transferredBytes)	On Recv 기능을 수행하는 함수입니다.
OnSend	void OnSend(int sessionIndex, DWORD transferredBytes)	On Send 기능을 수행하는 함수입니다.
SendProtocol	void SendProtocol(Session* session)	Protocol 데이터를 네트워크나 다른 시스템으로 전달합니다.
PacketProcess	void PacketProcess(int sessionIndex, int packetID, std::vector<char>& data)	Packet Process 기능을 수행하는 함수입니다.
DBWorkerThread	void DBWorkerThread()	DB Worker Thread 기능을 수행하는 함수입니다.
TestPacketProcessor	void IOCPServer::TestPacketProcessor()	Test Packet Processor 기능을 수행하는 함수입니다.
InitLogger	void IOCPServer::InitLogger()	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
Init	bool IOCPServer::Init(int port, int maxSessionCount)	초기 상태를 구성하고 필요한 객체나 데이터를 준비합니다.
Start	void IOCPServer::Start()	Start 기능을 수행하는 함수입니다.
AcceptThread	void IOCPServer::AcceptThread()	Accept Thread 기능을 수행하는 함수입니다.
WorkerThread	void IOCPServer::WorkerThread()	Worker Thread 기능을 수행하는 함수입니다.
OnConnect	void IOCPServer::OnConnect(Session* newSession, SOCKADDR_IN clientAddr)	On Connect 기능을 수행하는 함수입니다.
OnDisconnect	void IOCPServer::OnDisconnect(int sessionIndex)	On Disconnect 기능을 수행하는 함수입니다.
OnRecv	void IOCPServer::OnRecv(int sessionIndex, DWORD transferredBytes)	On Recv 기능을 수행하는 함수입니다.
WSAGetLastError	SOCKET_ERROR && WSAGetLastError() != ERROR_IO_PENDING)	WSA Get Last Error 기능을 수행하는 함수입니다.
OnSend	void IOCPServer::OnSend(int sessionIndex, DWORD transferredBytes)	On Send 기능을 수행하는 함수입니다.
SendPacket	void IOCPServer::SendPacket(int sessionIndex, int packetID, const char* data, int len)	Packet 데이터를 네트워크나 다른 시스템으로 전달합니다.

이름	시그니처	설명
SendProtocol	void IOCPServer::SendProtocol(Session* session)	Protocol 데이터를 네트워크나 다른 시스템으로 전달합니다.
PacketProcess	void IOCPServer::PacketProcess(int sessionIndex, int packetID, std::vector<char>& data)	Packet Process 기능을 수행하는 함수입니다.
DBWorkerThread	void IOCPServer::DBWorkerThread()	DB Worker Thread 기능을 수행하는 함수입니다.
PushDBTask	void IOCPServer::PushDBTask(DBData data)	Push DB Task 기능을 수행하는 함수입니다.
CreateRoomTry	void IOCPServer::CreateRoomTry(Session* Client)	Room Try 객체 또는 리소스를 생성합니다.
JoinRoomTry	void IOCPServer::JoinRoomTry(Session* Client, int roomID)	Room Try 흐름에 참가하거나 연결을 수립합니다.
GameStartTry	void IOCPServer::GameStartTry(Session* Client)	Game Start Try 기능을 수행하는 함수입니다.
BroadcastRoomList	void IOCPServer::BroadcastRoomList()	Broadcast Room List 기능을 수행하는 함수입니다.
BroadcastRoomInfo	void IOCPServer::BroadcastRoomInfo(Session* client)	Broadcast Room Info 기능을 수행하는 함수입니다.
GameFrameProtocol	void IOCPServer::GameFrameProtocol(float deltaTime)	Game Frame Protocol 기능을 수행하는 함수입니다.

session.h

- 절대 경로: `D:₩FinalProject₩UnrealConnectServer_last₩MainServer₩MainServer₩session.h`
- 파일 목적: 클라이언트 세션 상태, 이동 의도, 게임 데이터 저장 구조를 정의하는 파일입니다.
- 주요 타입: `OverlappedEx, Session`
- 함수 수(추출): 4
- 멤버 변수 수(추출): 20

멤버 변수

이름	타입	설명
type	IO_OPERATION	type 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
socket	SOCKET	socket 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
playerUID	int	세션/플레이어 식별자와 관련된 상태입니다.
sessionID	int	세션/플레이어 식별자와 관련된 상태입니다.
roomSlot	int	방 상태, 방 UI, 방 참가자와 관련된 멤버입니다.
teamID	int	teamID 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
isConnected	bool	isConnected 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
isReady	bool	isReady 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
LastMoveIntent	MoveIntent	LastMoveIntent 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
MoveIntentLock	std::mutex	MoveIntentLock 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
now_state	ESessionState	now_state 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
gameDatas	GameData	gameDatas 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
playerName	std::wstring	playerName 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
TempBuffer[1024]	char	TempBuffer[1024] 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

이름	타입	설명
recvBuffer	s_ringbuf	recvBuffer 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
recvOverlapped	OverlappedEx	recvOverlapped 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
isSending	bool	isSending 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
sendQueue	std::queue<std::vector<char>>	동일한 종류의 데이터를 순서대로 저장하는 컨테이너입니다.
sendLock	std::mutex	sendLock 기능에 필요한 런타임 상태 또는 참조를 저장합니다.
sendOverlapped	OverlappedEx	sendOverlapped 기능에 필요한 런타임 상태 또는 참조를 저장합니다.

함수

이름	시그니처	설명
ZeroMemory	ZeroMemory(TempBuffer, sizeof(TempBuffer))	Zero Memory 기능을 수행하는 함수입니다.
ZeroMemory	ZeroMemory(&recvBuffer, sizeof(s_ringbuf))	Zero Memory 기능을 수행하는 함수입니다.
ZeroMemory	ZeroMemory(&recvOverlapped, sizeof(OverlappedEx))	Zero Memory 기능을 수행하는 함수입니다.
ZeroMemory	ZeroMemory(&sendOverlapped, sizeof(OverlappedEx))	Zero Memory 기능을 수행하는 함수입니다.